

C Piscine

Rush00

Summary: This document contains the instructions for Rush00 of the C Piscine @ 42.

Version: 9.0

Contents

I	Foreword	2
II	Instructions	4
III	AI Instructions	6
IV	Main subject	8
V	Rush00	10
VI	Rush01	11
VII	Rush02	12
VIII	Rush03	13
IX	Rush04	14
X	Bonus Points	15
XI	Submission and peer-evaluation	16

Chapter I

Foreword

Here are the lyrics of a famous TV show theme for everyone to enjoy!

```
[Verse 1]
I wanna be the very best
Like no one ever was
To catch them is my real test
To train them is my cause

I will travel across the land
Searching far and wide
Each pokemon to understand
The power that's inside

[Chorus]
Pokemon! Gotta catch 'em all! It's you and me
I know it's my destiny,
Pokemon! Oh you're my best friend
In a world, we must defend
Pokemon! A heart so true
Our courage will pull us through,

You teach me and I'll teach you,
Pokemon! Gotta catch'em all

[Chorus]

Every challenge along the way
With courage I will face.
I will battle every day
To claim my rightful place.
Come with me,
The time is right,
There's no better team.
Arm in arm we'll win the fight!
It's always been our dream!
```

`[Chorus]`

I bet you were singing just now! But for now, let's stay focused; this subject has nothing to do with *Pocket Monsters*!

Chapter II

Instructions

- You must do the project with the imposed team. Meeting with your teammates is part of your tasks, using any appropriate means (Slack, email, phone if available, or directly in person).
- If you have really tried everything to reach one of your teammates, but they still remain unreachable, follow the instructions from your pedagogical team, if provided. The default procedure is to do the project with the available teammates, and discuss the situation during the evaluation. Even if the group leader is missing, you still have access to the submission directory.
- Your work must comply with the Norm. If you have bonus files/functions, they are included in the norm check, and you will receive a score of 0 if there is a norm error.
- You must handle errors coherently. You may either print an error message or simply return control to the user.
- Your project must be completed and pushed to the Git repository by the deadline displayed on the project's main page on the intranet.
- You must follow the submission procedure described at the end of this document, if provided.
- Your program must compile using `cc` with the following flags: -Wall -Wextra -Werror. If there is a moulinette, it will use the same compiler and flags.
- If your program does not compile, you will receive a score of `0`.
- The group will be automatically registered for the defense. You must attend your evaluation with all of your teammates. The purpose of the defense is to present and explain your work comprehensively.
- Do not cancel your evaluation; you will not get a second one.
- Each member of the group must be fully aware of all the details of the project. If you choose to split the workload, ensure you understand every part completed by other team members. This understanding may be verified during the evaluation.
- This document contains **five** different versions of the subject.

- Your team must complete one version, determined by the following calculation:
  - Take the numerical index (from 1 to 26) of the first letter of the team leader's login.
  - Compute that number modulo 5.
  - The result determines which subject to complete.
- Refer to the attached `groups_subject.txt` file for more details and examples.

Chapter III

AI Instructions

Context

The C Piscine is intense. It's your first big challenge at 42 — a deep dive into problem-solving, autonomy, and community.

During this phase, your main objective is to build your foundation — through struggle, repetition, and especially **peer-learning** exchange.

In the AI era, shortcuts are easy to find. However, it's important to consider whether your AI usage is truly helping you grow — or simply getting in the way of developing real skills.

The Piscine is also a human experience — and for now, nothing can replace that. Not even AI.

For a more complete overview of our stance on AI — as a learning tool, as part of the ICT curriculum, and as a growing expectation in the job market — please refer to the dedicated FAQ available on the intranet.

Main message

- Build strong foundations without shortcuts.
- Really develop tech & power skills.
- Experience real peer-learning, start learning how to learn and solve new problems.
- The learning journey is more important than the result.
- Learn about the risks associated with AI, and develop effective control practices and countermeasures to avoid common pitfalls.

Learner rules:

- You should apply reasoning to your assigned tasks, especially before turning to AI.
- You should not ask for direct answers to the AI.
- You should learn about 42 global approach on AI.

Phase outcomes:

Within this foundational phase, you will get the following outcomes:

- Get proper tech and coding foundations.
- Know why and how AI can be dangerous during this phase.

Comments and example:

- Yes, we know AI exists — and yes, it can solve your projects. But you're here to learn, not to prove that AI has learned. Don't waste your time (or ours) just to demonstrate that AI can solve the given problem.
- Learning at 42 isn't about knowing the answer — it's about developing the ability to find one. AI gives you the answer directly, but that prevents you from building your own reasoning. And reasoning takes time, effort, and involves failure. The path to success is not supposed to be easy.
- Keep in mind that during exams, AI is not available — no internet, no smartphones, etc. You'll quickly realise if you've relied too heavily on AI in your learning process.
- Peer learning exposes you to different ideas and approaches, improving your interpersonal skills and your ability to think divergently. That's far more valuable than just chatting with a bot. So don't be shy — talk, ask questions, and learn together!
- Yes, AI will be part of the curriculum — both as a learning tool and as a topic in itself. You'll even have the chance to build your own AI software. In order to learn more about our crescendo approach you'll go through in the documentation available on the intranet.

✓ Good practice:

> I'm stuck on a new concept. I ask someone nearby how they approached it. We talk for 10 minutes — and suddenly it clicks. I get it.

✗ Bad practice:

> I secretly use AI, copy some code that looks right. During peer evaluation, I can't explain anything. I fail. During the exam — no AI — I'm stuck again. I fail.

Chapter IV

Main subject

	Exercise00
Rush0X	
Directory: *ex*00/	
Files to Submit: `main.c, ft_putchar.c, rush0X.c`	
Authorized: `write`	

Your program must display a rectangle on the screen, following these requirements:

Files to submit:

- `main.c`
- `ft_putchar.c`
- `rush0X.c`, where "0X" represents the rush number (e.g., rush00.c).
    - The next chapters will describe the specific constraints for each rush number.
    - All files should be placed in the appropriate directory.

Compilation:

- These **three** files will be compiled together (see the **Instructions** chapter).
- The `ft_putchar.c` file must contain the `ft_putchar` function only.

Example of `main.c` file:

```
int        main()
{
        rush(5, 5);
        return (0);
}
```

Function requirements:

- You must write a function called `rush` with the following specifications:
  - It must take two integer arguments, named x and y.
  - The function must be placed in the rush0X.c file.
- The function must be prototyped as follows:

```
        void        rush(int x, int y);
```

- Your `rush` function must display a rectangle on the screen with a width of `x` characters and a height of `y` characters.
- Your function must never crash or enter an infinite loop.
- Your `main` function will be modified during the defense to check whether you have handled all required cases.
  Here is an example of a test that will be performed:

```
        int        main()
        {
                rush(123, 42);
                return (0);
        }
```

- Your function must handle any integer passed as a parameter, including zero and negative values, without crashing. You are free to choose the behavior in these cases (for example printing nothing or an error message), and you will explain it during the evaluation.

Chapter V

Rush00

- rush(5, 3) should display:

```
$>./a.out
o---o
|   |
o---o
$>
```

- rush(5, 1) should display:

```
$>./a.out
o---o
$>
```

- rush(1, 1) should display:

```
$>./a.out
o
$>
```

- rush(1, 5) should display:

```
$>./a.out
o
|
|
|
o
$>
```

- rush(4, 4) should display:

```
$>./a.out
o--o
|  |
|  |
o--o
$>
```

Chapter VI

Rush01

- rush(5, 3) should display:

```
$>./a.out
/***\
*   *
\***/
$>
```

- rush(5, 1) should display:

```
$>./a.out
/***\
$>
```

- rush(1, 1) should display:

```
$>./a.out
/
$>
```

- rush(1, 5) should display:

```
$>./a.out
/
*
*
*
\
$>
```

- rush(4, 4) should display:

```
$>./a.out
/**\
*  *
*  *
\**/
$>
```

Chapter VII

Rush02

- rush(5, 3) should display:

```
$>./a.out
ABBBA
B   B
CBBBC
$>
```

- rush(5, 1) should display:

```
$>./a.out
ABBBA
$>
```

- rush(1, 1) should display:

```
$>./a.out
A
$>
```

- rush(1, 5) should display:

```
$>./a.out
A
B
B
B
C
$>
```

- rush(4, 4) should display:

```
$>./a.out
ABBA
B  B
B  B
CBBC
$>
```

Chapter VIII

Rush03

- rush(5, 3) should display:

```
$>./a.out
ABBBC
B   B
ABBBC
$>
```

- rush(5, 1) should display:

```
$>./a.out
ABBBC
$>
```

- rush(1, 1) should display:

```
$>./a.out
A
$>
```

- rush(1, 5) should display:

```
$>./a.out
A
B
B
B
A
$>
```

- rush(4, 4) should display:

```
$>./a.out
ABBC
B  B
B  B
ABBC
$>
```

Chapter IX

Rush04

- rush(5, 3) should display:

```
$>./a.out
ABBBC
B   B
CBBBA
$>
```

- rush(5, 1) should display:

```
$>./a.out
ABBBC
$>
```

- rush(1, 1) should display:

```
$>./a.out
A
$>
```

- rush(1, 5) should display:

```
$>./a.out
A
B
B
B
C
$>
```

- rush(4, 4) should display:

```
$>./a.out
ABBC
B  B
B  B
CBBA
$>
```

Chapter X

Bonus Points

You can earn bonus points by submitting other versions of the project with their corresponding names (e.g. `rush01.c`, `rush02.c`, ...). These additional files must be located with the other files, in the same directory. For each different rush0X.c file, the program will be recompiled.

It is also possible to create an enhanced main function, in a `main_bonus.c` file, that takes two command-line arguments representing the two parameters for the `rush()` function. For this bonus only, you may use the standard `atoi()` function, or recode it. Reject invalid arguments with an error message.



Make sure your assigned subject works perfectly before attempting any bonuses!
If your bonus submission works, but your original assignment fails, you will receive a score of 0.

Chapter XI

Submission and peer-evaluation

Submit your assignment to your `Git` repository as usual. Only the work inside your repository will be evaluated during the defense. Make sure to double-check the names of your files to ensure they are correct.

This assignment is not automatically verified by a program.



`You must submit only the files explicitly requested in the project instructions, in the appropriate place.`